

■ LEVEL 22 • GRAPHS

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Entities connected by edges. First classify directed/undirected, weighted/unweighted, cyclic/acyclic.
Reference Examples	Adjacency List/Matrix • Number of Islands • Clone Graph • Course Schedule • DSU • Dijkstra
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 22 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Number of Islands	Graph / Grid DFS	#200
2	Clone Graph	Graph DFS	#133
3	Course Schedule	Directed Graph / Cycle Detection	#207
4	Redundant Connection	DSU / Graph	#684
5	Network Delay Time	Weighted Graph + Dijkstra	#743
6	Graph Valid Tree	Graph + DSU	#261
7	Number of Islands	Grid Graph	200
8	Clone Graph	Graph Traversal + HashMap	133
9	Course Schedule	Directed Graph + Topological Sort	207
10	Redundant Connection	DSU / Union Find	684

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Number of Islands

Pattern: Graph / Grid DFS

LeetCode: #200

■ Input	2D grid of land/water
■ Output	number of connected islands
■ Constraints	grid cells are 0/1
■ Main Idea / Pattern	Each unvisited land cell starts one component. DFS marks its whole component.
■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
void dfs(int r, int c, vector<vector<char>>& g) {
    if (r < 0 || r >= g.size() ||
        c < 0 || c >= g[0].size() ||
        g[r][c] != '1')
        return;

    g[r][c] = '0';

    dfs(r + 1, c, g);
    dfs(r - 1, c, g);
    dfs(r, c + 1, g);
    dfs(r, c - 1, g);
}

int numIslands(vector<vector<char>>& grid) {
    int ans = 0;

    for (int r = 0; r < grid.size(); r++)
        for (int c = 0; c < grid[0].size(); c++)
            if (grid[r][c] == '1') {
                ans++;
                dfs(r, c, grid);
            }

    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Clone Graph

Pattern: Graph DFS

LeetCode: #133

■ Input	connected graph node
■ Output	deep copy of graph
■ Constraints	node values unique
■ Main Idea / Pattern	Map each original node to its clone. Recursively clone neighbors.
■■ Time Complexity	$O(V+E)$
■ Space Complexity	$O(V)$

■ Optimal C++ Code

```
unordered_map<Node*, Node*> mp;

Node* clone(Node* node) {
    if (!node) return nullptr;
    if (mp.count(node)) return mp[node];

    Node* copy = new Node(node->val);
    mp[node] = copy;

    for (Node* nei : node->neighbors)
        copy->neighbors.push_back(clone(nei));

    return copy;
}

Node* cloneGraph(Node* node) {
    return clone(node);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Course Schedule

Pattern: Directed Graph / Cycle Detection

LeetCode: #207

■ Input	number of courses and prerequisites
■ Output	whether all courses can be completed
■ Constraints	prerequisites form directed edges
■ Main Idea / Pattern	A course dependency cycle makes completion impossible. Use indegrees and Kahn's algorithm.
■ Time Complexity	$O(V+E)$
■ Space Complexity	$O(V+E)$

■ Optimal C++ Code

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> g(numCourses);
    vector<int> indeg(numCourses);

    for (auto& p : prerequisites) {
        g[p[1]].push_back(p[0]);
        indeg[p[0]]++;
    }

    queue<int> q;
    for (int i = 0; i < numCourses; i++)
        if (indeg[i] == 0) q.push(i);

    int done = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        done++;

        for (int v : g[u])
            if (--indeg[v] == 0)
                q.push(v);
    }

    return done == numCourses;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Redundant Connection

Pattern: DSU / Graph

LeetCode: #684

■ Input	undirected edges
■ Output	edge whose removal makes the graph a tree
■ Constraints	one extra edge creates a cycle
■ Main Idea / Pattern	Union endpoints. The first edge whose endpoints already share a component is redundant.
■ Time Complexity	$O(n \alpha(n))$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
class DSU {
    vector<int> p, sz;
public:
    DSU(int n) : p(n + 1), sz(n + 1, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
        return true;
    }
};

vector<int> findRedundantConnection(vector<vector<int>>& edges) {
    DSU dsu(edges.size());
    for (auto& e : edges)
        if (!dsu.unite(e[0], e[1]))
            return e;
    return {};
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Network Delay Time

Pattern: Weighted Graph + Dijkstra

LeetCode: #743

■ Input	directed weighted edges and source
■ Output	time for all nodes to receive signal
■ Constraints	edge weights positive
■ Main Idea / Pattern	Use Dijkstra because all edge weights are positive. Finalize the smallest tentative distance first.
■ Time Complexity	$O((V+E) \log V)$
■ Space Complexity	$O(V+E)$

■ Optimal C++ Code

```
int networkDelayTime(vector<vector<int>>& times, int n, int k) {
    vector<vector<pair<int, int>>> g(n + 1);

    for (auto& e : times)
        g[e[0]].push_back({e[1], e[2]});

    const int INF = 1e9;
    vector<int> dist(n + 1, INF);
    priority_queue<pair<int, int>,
                  vector<pair<int, int>>,
                  greater<pair<int, int>>> pq;

    dist[k] = 0;
    pq.push({0, k});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();

        if (d != dist[u]) continue;

        for (auto [v, w] : g[u]) {
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }

    int ans = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == INF) return -1;
        ans = max(ans, dist[i]);
    }

    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Graph Valid Tree

Pattern: Graph + DSU

LeetCode: #261

■ Input	n nodes and undirected edges
■ Output	whether edges form exactly one tree
■ Constraints	must have n-1 edges
■ Main Idea / Pattern	A tree needs n-1 edges and no cycle. DSU detects cycles while merging.
■ Time Complexity	$O(n \alpha(n))$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
bool validTree(int n, vector<vector<int>>& edges) {
    if (edges.size() != n - 1)
        return false;

    vector<int> p(n), sz(n, 1);
    iota(p.begin(), p.end(), 0);

    function<int(int)> find = [&](int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    };

    for (auto& e : edges) {
        int a = find(e[0]), b = find(e[1]);

        if (a == b) return false;

        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a;
        sz[a] += sz[b];
    }

    return true;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Number of Islands

Pattern: Grid Graph

LeetCode: 200

Input	grid
Output	number of connected land components
Constraints	rows, cols <= 300
Main Idea / Pattern	Treat each land cell as a graph node and flood-fill each unvisited component.
Time Complexity	O(mn)
Space Complexity	O(mn)

Optimal C++ Code

```
class Solution {
public:
    int numIslands(vector<vector<char>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        int ans = 0;
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] != '1') continue;
                ans++;
                queue<pair<int, int>> q;
                q.push({i, j});
                grid[i][j] = '0';
                while (!q.empty()) {
                    auto [r, c] = q.front();
                    q.pop();
                    int dr[] = {-1, 1, 0, 0};
                    int dc[] = {0, 0, -1, 1};
                    for (int d = 0; d < 4; d++) {
                        int nr = r + dr[d];
                        int nc = c + dc[d];
                        if (nr >= 0 && nr < m &&
                            nc >= 0 && nc < n &&
                            grid[nr][nc] == '1') {
                            grid[nr][nc] = '0';
                            q.push({nr, nc});
                        }
                    }
                }
            }
        }
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Clone Graph

Pattern: Graph Traversal + HashMap

LeetCode: 133

Input	node
Output	deep copy of a connected graph
Constraints	0 <= nodes <= 100
Main Idea / Pattern	Map each original node to its clone and recursively clone all neighbors.
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

Optimal C++ Code

```
class Solution {
    unordered_map<Node*, Node*> mp;
public:
    Node* cloneGraph(Node* node) {
        if (!node) return nullptr;
        if (mp.count(node)) return mp[node];
        Node* copy = new Node(node->val);
        mp[node] = copy;
        for (Node* nei : node->neighbors) {
            copy->neighbors.push_back(cloneGraph(nei));
        }
        return copy;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Course Schedule

Pattern: Directed Graph + Topological Sort

LeetCode: 207

Input	numCourses, prerequisites
Output	whether all courses can be completed
Constraints	1 <= courses <= 2000
Main Idea / Pattern	A cycle in the prerequisite graph makes completion impossible; detect it with indegrees and BFS.
Time Complexity	$O(V + E)$
Space Complexity	$O(V + E)$

Optimal C++ Code

```
class Solution {
public:
    bool canFinish(int n, vector<vector<int>>& prerequisites) {
        vector<vector<int>> g(n);
        vector<int> indegree(n);

        for (auto& p : prerequisites) {
            g[p[1]].push_back(p[0]);
            indegree[p[0]]++;
        }

        queue<int> q;
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) q.push(i);
        }

        int done = 0;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            done++;

            for (int v : g[u]) {
                if (--indegree[v] == 0) {
                    q.push(v);
                }
            }
        }

        return done == n;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Redundant Connection

Pattern: DSU / Union Find

LeetCode: 684

Input	edges
Output	edge whose removal makes the graph a tree
Constraints	n edges on n nodes
Main Idea / Pattern	If an edge connects two nodes already in the same component, it is redundant.
Time Complexity	$O(n \alpha(n))$
Space Complexity	$O(n)$

Optimal C++ Code

```
class Solution {
    vector<int> parent, size;

    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
        return true;
    }

public:
    vector<int> findRedundantConnection(vector<vector<int>>& edges) {
        int n = edges.size();
        parent.resize(n + 1);
        size.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
        for (auto& e : edges) {
            if (!unite(e[0], e[1])) return e;
        }
        return {};
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.